Big-Data Driven Taxi Fare Prediction and Demand Hotspot Analytics

Aditya S
School of Artificial Intelligence
Amrita Vishwa Vidyapeetham
Coimbatore, India
CB.AI.U4AID24104

Ashwin Naresh M
School of Artificial Intelligence
Amrita Vishwa Vidyapeetham
Coimbatore, India
CB.AI.U4AID24106

Divya Rithanya S
School of Artificial Intelligence
Amrita Vishwa Vidyapeetham
Coimbatore, India
CB.AI.U4AID24113

Vivethasri C
School of Artificial Intelligence
Amrita Vishwa Vidyapeetham
Coimbatore, India
CB.AI.U4AID24162

Dr. Sreeja B P
Assistant Professor Sr.Gd., School of AI
Amrita Vishwa Vidyapeetham
Coimbatore, India

***Abstract*—Urban ride-hailing networks generate massive, continuous streams of spatial mobility events characterized by extreme non-linear price volatility, spatiotemporal demand imbalances, and severe outlier noise. Traditional centralized database systems and single-machine analytics scripts fail to ingest, clean, and model such velocity and volume within actionable operational latencies. This paper presents an end-to-end, big data analytics and stream processing platform engineered on Hadoop HDFS and Apache Spark with Scala. The batch processing layer stores over 33.5 Million NYC Yellow Taxi trip records and hourly meteorological observations (45.3 GB replicated across a multi-node HDFS cluster) and executes distributed outlier pruning, spatiotemporal enrichment, and MLlib model training. A 19-feature L_2-regularized Ridge Regression model trained via L-BFGS on 26.8 Million historical records achieves $R^2 = 0.7998$ and a Mean Absolute Error of $4.40. For fleet allocation, an unsupervised K-Means clustering pipeline applying a $\log(1+x)$ variance-stabilizing transform isolates three distinct operational demand regimes, elevating the cluster Silhouette score from 0.3898 to 0.6399 (+64%). The speed layer leverages Spark Structured Streaming with 3-second micro-batches, 1-hour stateful sliding windows, and 10-minute event-time watermarking to execute dual-branch live fare scoring and real-time hotspot classification. Stream outputs are buffered to HDFS and rendered through a WebHDFS REST bridge to an interactive Leaflet GIS dashboard on a locally hosted web interface. This paper presents an exhaustive residual error slicing, empirical execution benchmarks, and system limitations.**

***Index Terms*—big data analytics, Apache Spark, Hadoop HDFS, Spark MLlib, Structured Streaming, Ridge regression, K-Means clustering, urban mobility**

I. Introduction

Urban transportation systems in metropolitan hubs such as New York City represent complex cyber-physical environments where hundreds of thousands of vehicular hailing events occur daily across dense spatial zones [1]. Efficient fleet dispatch, congestion mitigation, and fair fare estimation require processing millions of historical trips while simultaneously ingesting and scoring live trip requests [2]. However, metered taxi pricing exhibits high non-linearity due to dynamic congestion, airport access tolls, severe weather, and diurnal demand spikes. Furthermore, spatial demand distribution is drastically right-skewed: drivers routinely cluster in hyper-dense transit corridors (e.g., Midtown Manhattan and JFK Airport), leaving outer borough zones chronically underserved [6].

Addressing these dual challenges requires modern big data infrastructure. Analyzing multi-year trip archives exceeds single-machine physical memory and CPU boundaries. For instance, parsing 33.5 Million trips across 263 geographic zones involves evaluating millions of spatiotemporal bins and generating substantial intermediate shuffle records during aggregation. Simultaneously, streaming operational decisions demand low latency, robust state recovery, and bounded memory growth to prevent out-of-memory failures under high velocity.

This paper describes the architecture, implementation, and empirical evaluation of an end-to-end distributed platform for taxi fare estimation and demand hotspot analytics. Our key contributions are:

1) **End-to-End Hybrid Architecture:** A Lambda-style architecture combining a 45.3 GB replicated HDFS batch layer with a 3-second Spark Structured Streaming speed layer, sharing a unified Scala 2.12 codebase on Apache Spark 3.5.6.
2) **Multi-Modal Ingestion & Quality Auditing:** Scalable ingestion uniting Snappy-compressed Parquet trip logs from the NYC TLC data source [1], Open-Meteo weather data [14], and spatial GIS data with strict physical boundary filtering (0.1–100 miles, $3.00–$500.00).
3) **Supervised Distributed Fare Regression:** Assembly of a 19-feature vector pipeline evaluated with L-BFGS Ridge Regression over 26.8 Million training trips, yielding $R^2 = 0.7998$ and MAE = $4.40 on 6.7 Million chronologically held-out test trips.

4) **Log-Transformed Hotspot Discovery:** Formulation of an unsupervised K-Means ($K = 3$) clustering methodology using a $\log(1 + x)$ variance-stabilizing transform that boosts the Silhouette score from 0.3898 to 0.6399.
5) **Fault-Tolerant Stream Inference & Visual Dashboard:** Implementation of dual-branch micro-batch streaming with 10-minute watermarks, coupled with a WebHDFS REST bridge and an interactive Leaflet GIS dashboard.

The remainder of this paper is structured as follows. Section II reviews related literature. Section III presents the problem formulation, 5 V characteristics, and multi-modal datasets. Section IV details the system architecture and cluster setup. Section V elaborates the batch, machine learning, and streaming implementations. Section VI defines the experimental methodology, Section VII discusses quantitative results and error slicing, Section VIII outlines engineering limitations, and Section IX concludes.

II. Related Work

A. Machine Learning for Fare Prediction

Taxi fare estimation has been studied using both regularized linear models and non-linear machine-learning approaches. Agarwal *et al.* [4] evaluated multiple regression approaches including Random Forest, Lasso Regression, XGBoost, and Ridge Regression for taxi fare prediction. Regularized linear models are particularly useful when correlated predictors are present because L_1 or L_2 penalties can constrain coefficient magnitude and reduce overfitting.

Sundravadivelu *et al.* [3] investigated Random Forest-based taxi fare prediction using pickup/drop-off locations and ride attributes. Tree ensembles can capture non-linear relationships between trip characteristics, although their computational characteristics differ from linear models used for high-throughput inference. In the present system, Ridge Regression is selected to provide a compact distributed model with inexpensive pointwise inference. The Ridge objective is optimized using L-BFGS, a limited-memory quasi-Newton method developed for large-scale optimization [13].

B. Spatiotemporal Demand & Hotspot Clustering

Urban taxi-demand analysis requires both spatial and temporal information. Mishra and Khetarpaul [6] investigated taxi hotspot identification in spatiotemporal space and highlighted the imbalance between pickup demand and taxi supply across urban regions.

Ramadhan *et al.* [5] applied K-Means clustering to a large NYC taxi trip dataset and used preprocessing, normalization, the Elbow Method, and Silhouette Score to investigate urban transportation patterns. These findings motivate the use of clustering for identifying operational demand regimes.

In the present work, pickup counts are strongly right-skewed. Therefore, a $\log(1 + x)$ transformation is applied before distance-based clustering to reduce the influence of extreme high-volume observations.

C. Big Data Frameworks & Stream Processing

Distributed architectures are essential for large-scale mobility analytics. The Hadoop Distributed File System (HDFS) provides a distributed storage architecture designed for large datasets and fault-tolerant operation across commodity hardware [7].

Apache Spark introduced Resilient Distributed Datasets (RDDs), providing a fault-tolerant distributed-memory abstraction for cluster computation [8]. Spark SQL subsequently introduced relational processing through DataFrames and the Catalyst query optimizer [9]. MLlib extends Spark with scalable machine-learning functionality and pipeline abstractions [11].

For real-time processing, Structured Streaming provides a declarative stream-processing API integrated with Spark's relational engine and supports event-time processing and triggers [10]. The broader batch/speed separation used in this project follows the Lambda Architecture principles described by Marz and Warren [12].

D. Research Gap

Prior mobility studies often concentrate on either offline predictive modeling or hotspot analysis independently. The present work integrates historical batch processing, fare regression, demand clustering, simulated streaming inference, HDFS-backed storage, and a GIS-based visualization layer in one reproducible environment.

III. Problem Formulation and Dataset

A. Problem Statement and Objective

The project addresses three principal urban-mobility challenges:

- **Urban mobility:** Large-scale taxi hailing activity exhibits high volatility across 263 NYC taxi zones.
- **Price uncertainty:** Fares vary with trip kinematics, temporal conditions, spatial characteristics, and environmental conditions.
- **Demand imbalance:** High-demand hubs attract a disproportionate share of activity while lower-demand zones may remain underserved.

Given an archive of historical taxi trips $\mathcal{D}_{\text{trips}}$, hourly weather observations $\mathcal{D}_{\text{weather}}$, and spatial zone information $\mathcal{D}_{\text{zones}}$, the objective is to build a fault-tolerant distributed pipeline that:

1) cleans, joins, and normalizes multi-modal data at scale;
2) trains a regularized regression model to predict metered fare $y \in \mathbb{R}^+$ from pre-ride kinematic, environmental, operational, temporal, and spatial features $\mathbf{x} \in \mathbb{R}^{19}$;
3) partitions citywide pickup intensity into discrete demand regimes using unsupervised clustering; and
4) executes continuous streaming inference on replayed trip streams using 3-second micro-batches while maintaining bounded state.

B. The Five V's of Big Data

The project exhibits the classic 5 V characteristics of big data:

- **Volume:** 33,484,865 trip records requiring 45.3 GB of replicated storage across Hadoop HDFS with 128 MB block sizing.
- **Velocity:** Ingestion and scoring of replayed trip micro-batches arriving every 3 seconds with 1-hour rolling stateful aggregations.
- **Variety:** Unification of columnar Snappy Parquet trip records, weather observations, and CSV/GeoJSON GIS information.
- **Veracity:** Multi-stage data hygiene enforcing physical bounds ($\$3.00 \leq$ fare $\leq \$500.00$; $0.1 \leq$ distance ≤ 100 mi) and chronological train/test splitting.
- **Value:** Fare prediction and data-driven fleet-dispatch support through demand-regime identification.

C. Historical Multi-Modal Dataset

The primary mobility dataset consists of monthly NYC Taxi and Limousine Commission (TLC) trip records. The official TLC repository publishes trip records with fields including pickup and drop-off timestamps, locations, trip distances, itemized fares, rate types, payment types, and passenger counts.

For this project, 53 monthly batches from January 2022 through May 2026 were incorporated into the historical processing pipeline.

Environmental variables were obtained using the Open-Meteo Historical Weather API. Open-Meteo provides historical reanalysis data, including hourly temperature, precipitation, weather code, and wind-speed variables. The project uses 38,712 hourly weather observations associated with the New York study area.

Geographic zone information is derived from the official TLC Taxi Zone Lookup Table and taxi-zone spatial data distributed through the TLC data portal.

D. Mathematical Formulation

Fare Regression: Let $\mathbf{x}_i \in \mathbb{R}^p$ denote the 19-dimensional feature vector for trip i, and let $y_i \in \mathbb{R}$ denote the metered fare. The parameters $\mathbf{w} \in \mathbb{R}^p$ and bias $b \in \mathbb{R}$ are estimated by minimizing the regularized mean squared error:

$$\min_{\mathbf{w},b} \frac{1}{2N} \sum_{i=1}^{N} \left(\mathbf{w}^T \mathbf{x}_i + b - y_i\right)^2 + \frac{\lambda}{2} \|\mathbf{w}\|_2^2, \tag{1}$$

where $\lambda > 0$ denotes the L_2 regularization parameter. The optimization is performed using L-BFGS, a limited-memory quasi-Newton method designed for large-scale optimization.

Hotspot Demand Regimes: Given hourly pickup volume $d_{z,t}$ for zone z at hour $t \in [0, 23]$, the variance-stabilized feature vector is defined as

$$\mathbf{z}_{z,t} = \begin{bmatrix} \ln(1 + d_{z,t}) \\ t \end{bmatrix} \in \mathbb{R}^2. \tag{2}$$

The observations are partitioned into $K = 3$ clusters $\{C_1, C_2, C_3\}$ by minimizing the within-cluster sum of squared Euclidean distances:

$$\min_{\{\boldsymbol{\mu}_k\}} \sum_{k=1}^{K} \sum_{\mathbf{z} \in C_k} \|\mathbf{z} - \boldsymbol{\mu}_k\|_2^2 . \tag{3}$$

E. Live Stream Simulation

To validate real-time inference without access to a proprietary live dispatch API, a Python replay generator (`replay_producer.py`) reads held-out March 2026 records from HDFS. It orders the records chronologically by pickup timestamp and emits structured micro-batches containing approximately 200–250 trip records every 3 seconds into

`/surgesense/streaming/incoming/`.

IV. System Architecture

The system follows the hybrid batch–stream architecture shown in Fig. 1. HDFS provides distributed storage, Spark performs batch model training and real-time inference, and WebHDFS serves the resulting analytics to the interactive dashboard.

A. Storage: Replicated HDFS Substrate

Hadoop HDFS 3.4.3 serves as the primary storage substrate. The cluster comprises one NameNode handling filesystem namespace metadata and two DataNodes storing partitioned file blocks. Storage blocks are configured at 128 MB with a replication factor of 2.

Raw trip archives reside under `/surgesense/raw/`, cleaned training/test partitions under `/surgesense/processed/`, serialized models under `/surgesense/models/`, and live streaming sinks under `/surgesense/streaming/`.

B. Distributed Execution: Apache Spark Engine

Apache Spark 3.5.6 executes batch preprocessing, model training, and streaming workflows. Spark SQL uses the Catalyst optimizer to transform relational expressions into optimized execution plans. ML operations use Spark MLlib.

V. Implementation

A. Batch Layer: Parsing, Cleaning & Enrichment

Raw Parquet files are read and transformed through three sequential stages:

1) **Cleaning :** Eliminates trips with non-positive duration, speeds $>$ 100 mph, passenger counts outside $[1, 6]$, distances outside $[0.1, 100.0]$ miles, and metered fares outside $[\$3.00, \$500.00]$.
2) **Normalization :** Standardizes timestamp handling and casts identifier columns into the required data types.
3) **Spatiotemporal Enrichment :** Joins trip records with taxi-zone information and hourly weather observations using the corresponding spatial and temporal keys.

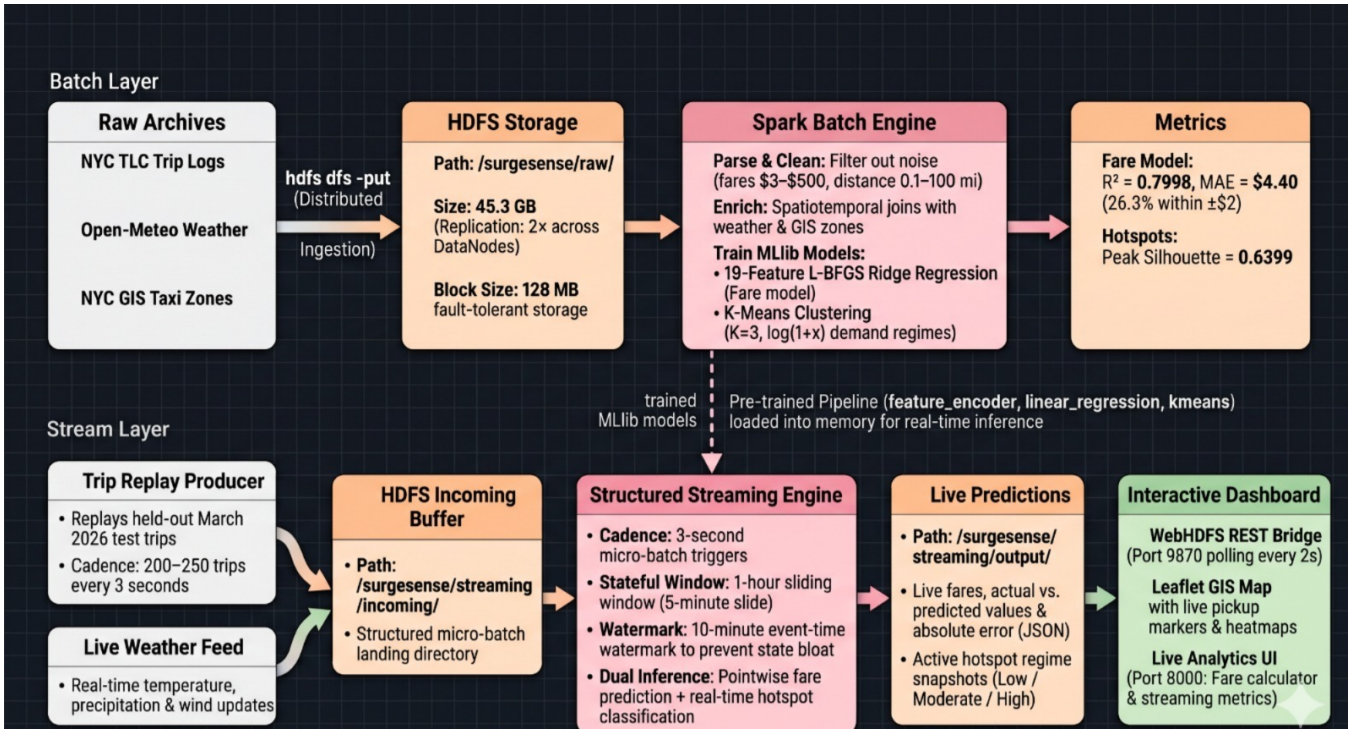


Fig. 1: End-to-end system architecture integrating HDFS storage, Spark batch model training, Structured Streaming inference, and the interactive dashboard.

TABLE I: System Hardware and Software Specifications

Component	Deployment Specification
Storage Substrate	Hadoop HDFS 3.4.3 (1 NameNode, 2 DataNodes)
HDFS Replication	Replication Factor = 2; Block Size = 128 MB
Compute Engine	Apache Spark 3.5.6 (Master + Worker, 8 Cores)
Programming Runtime	Scala 2.12.18, Java JDK 11, SBT 1.9.8
ML Library	Spark MLlib (`org.apache.spark.ml`)
Streaming Engine	Spark Structured Streaming (3-second micro-batches)
Driver Memory	6.0 GB RAM allocated to Spark JVM
Shuffle Partitions	`spark.sql.shuffle.partitions = 8`
Serving Bridge	Python 3.10 WebHDFS REST Client (Port 9870)
User Interface	Interactive Leaflet GIS Dashboard (Localhost)

B. Batch Layer: Feature Engineering

A total of 19 predictors are assembled through Spark's `VectorAssembler`:

- **Kinematics (3):** `trip_distance`, derived `trip_duration_min`, and `average_speed_mph`.
- **Temporal Cyclicality (4):** `pickup_hour`, `day_of_week`, `is_weekend`, and `pickup_month`.
- **Environmental Metrics (4):** `temperature_2m`, `precipitation`, `wind_speed_10m`, and `weather_code`.
- **Spatial One-Hot Vectors (5):** Borough indicators for Manhattan, Queens, Brooklyn, Bronx, and Staten Island.
- **Operational Attributes (3):** `passenger_count`, `rate_code_id`, and `payment_type`.

Continuous features are standardized before model training.

C. Speed Layer: Spark Structured Streaming Engine

The streaming inference engine (`StreamingInference.scala`) reads micro-batches from `/surgesense/streaming/incoming/`. Structured Streaming is designed around incremental execution of declarative streaming queries and supports event-time processing and triggers. The project executes two concurrent branches:

- **Branch 1 — Pointwise Fare Inference:** Incoming trip events are passed through the pre-trained `PipelineModel` loaded from HDFS. The regression model evaluates

$$\hat{y} = \mathbf{w}^T \mathbf{x} + b \tag{4}$$

for each ride. The resulting stream contains actual fare, predicted fare, and absolute residual $|y - \hat{y}|$.
- **Branch 2 — Stateful Hotspot Aggregation:** A 10-minute event-time watermark is applied and pickup events are aggregated over a 1-hour sliding window updated every 5 minutes:

$$\text{Window} = [t - 60 \text{ min}, t], \tag{5}$$

$$\text{Watermark} = \max(t_{\text{event}}) - 10 \text{ min}. \tag{6}$$

The resulting pickup counts are transformed using Eq. (2) and classified by the pre-trained K-Means model into Low-, Moderate-, or High-Demand regimes.

D. Serving & Dashboard Visualization

Stream predictions and aggregated hotspot regimes are committed to HDFS using `foreachBatch` sinks. A background service (`bridge.py`) polls WebHDFS endpoints at 2-second intervals and aggregates live records into a JSON telemetry snapshot.

The locally hosted front-end provides multiple analytical views. Tab 1 contains the interactive fare calculator and route-cost output. Tab 2 provides demand-regime classification and an interactive Leaflet GIS visualization. Tab 4 presents real-time streaming telemetry, fare residuals, pickup locations, and active windowed demand regimes.

VI. Experimental Setup

All experiments were conducted in the Docker environment summarized in Table I.

- **Batch Training:** 33,484,865 records were split chronologically into 26,787,892 training trips (80%, Jan. 2022–Feb. 2026) and 6,696,973 held-out test trips (20%, Mar. 2026–May 2026).
- **Performance Metrics:** Fare prediction was evaluated using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), Coefficient of Determination (R^2), and $\pm\$2.00$ Tolerance Accuracy:

$$\text{Tol}_{\pm\$2.00} = \frac{1}{N}\sum_{i=1}^{N}\mathbb{I}\left(|y_i - \hat{y}_i| \le 2.00\right) \times 100\%. \tag{7}$$

- **Clustering Evaluation:** Cluster separation was evaluated using the Silhouette Coefficient across $K \in [2, 8]$.

TABLE II: Supervised Fare Regression Performance: Baseline vs. Tuned

Evaluation Metric	Baseline ($\lambda = 0.01$)	Tuned ($\lambda = 0.10$)
Coefficient of Determination (R^2)	0.799582	**0.799821**
Root Mean Squared Error (RMSE)	\$7.7935	**\$7.7889**
Mean Absolute Error (MAE)	\$4.4118	**\$4.4027**
Tolerance Accuracy ($\le \pm\$2.00$)	26.0683%	**26.3034%**
Total Absolute Within-Tol. Trips	1,745,785	**1,761,438**
Optimizer Solver	L-BFGS	L-BFGS
Training Time	184.2 s	181.6 s

VII. Results and Discussion

A. Supervised Fare Regression Performance

Table II summarizes the performance of the regularized Ridge Regression model evaluated on the 6.7 Million held-out test trips. Increasing the regularization parameter from $\lambda = 0.01$ to $\lambda = 0.10$ produces a small improvement across the reported metrics, achieving $R^2 = 0.799821$, RMSE = \$7.7889, and MAE = \$4.4027.

The tuned configuration places 26.3034% of test predictions within $\pm\$2.00$ of the corresponding metered fare, representing 1,761,438 held-out trips.

B. Unsupervised Demand Hotspots: K-Means Clustering

The Silhouette coefficient was evaluated for cluster configurations $K \in [2, 8]$. Direct clustering of the baseline feature representation reaches a peak Silhouette score of 0.3898. After applying the $\log(1 + x)$ transformation to pickup volume, the refined feature space reaches a Silhouette score of 0.6399 at $K = 3$.

The relative improvement is

$$\frac{0.6399 - 0.3898}{0.3898} \times 100 \approx 64.16\%. \tag{8}$$

Thus, $K = 3$ was retained for the Low-, Moderate-, and High-Demand operational regimes.

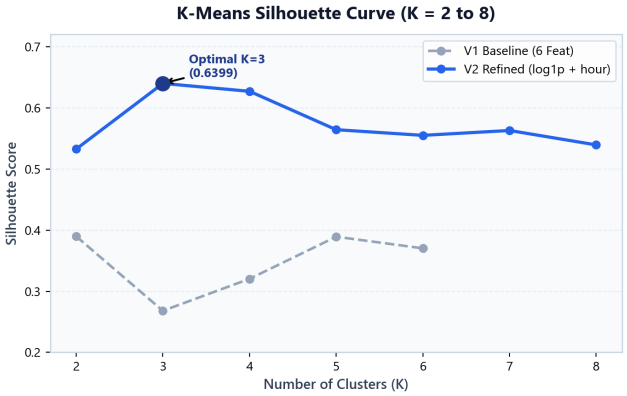
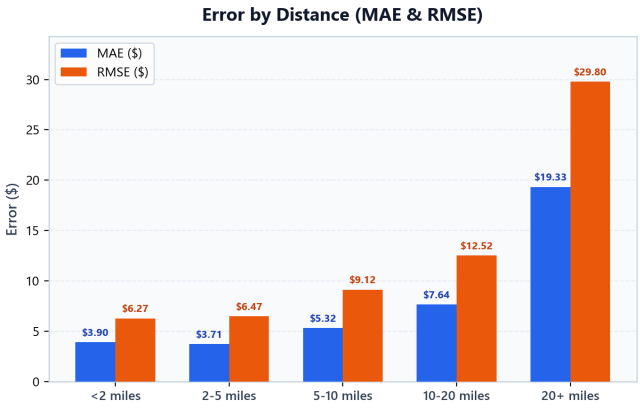


Fig. 2: K-Means Silhouette coefficient across cluster counts $K \in [2, 8]$. The $\log(1 + x)$ transformed feature space (V2 Refined) achieves a peak Silhouette score of 0.6399 at $K = 3$, compared with 0.3898 for the V1 baseline.

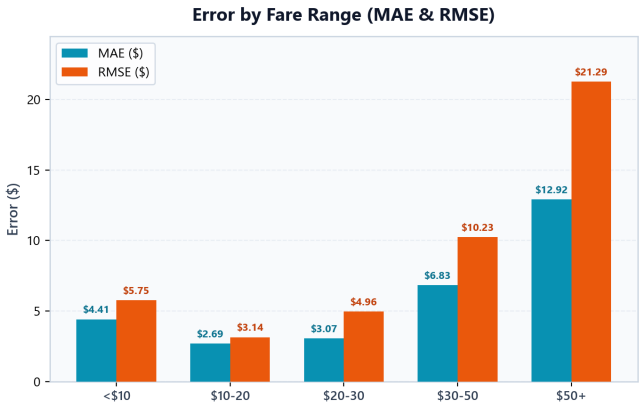
C. Residual Error Slicing Analysis

To identify the operating regions in which the fare model performs well or poorly, prediction errors were sliced across travel-distance, fare, and temporal intervals. The corresponding distance and fare results are presented in Fig. 3.

- **Distance Slicing:** Trips between 2–5 miles achieve MAE = \$3.71, while trips below 2 miles achieve MAE = \$4.27. Long-haul journeys above 20 miles exhibit substantially higher error, with MAE = \$19.33 and RMSE = \$29.80.
- **Fare Slicing:** Trips in the \$10–\$20 range exhibit MAE = \$2.69 and RMSE = \$3.14. Trips exceeding \$50 exhibit RMSE = \$21.29.
- **Diurnal Slicing:** Midday observations between 10:00 and 14:00 produce MAE = \$3.94, whereas the 05:00 period reaches MAE = \$6.50.



(a) Residual MAE and RMSE across distance intervals.



(b) Residual MAE and RMSE across fare ranges.

Fig. 3: Residual error slicing across travel-distance and metered-fare tiers on the held-out test set. Core short-to-medium-distance trips exhibit substantially lower error than the longest trips.

D. Spatiotemporal Demand Dynamics

The aggregated historical data also reveal distinct temporal and spatial demand patterns, as illustrated in Fig. 4. The 24-hour citywide demand curve exhibits a trough around 05:00 at approximately 81.0 trips/hr and rises toward an evening peak around 18:00 at approximately 142.9 trips/hr.

Spatial demand is concentrated in several high-volume taxi zones. In the project aggregation, Upper East Side South, JFK Airport, and Midtown Center are among the highest-volume pickup zones.

E. Dashboard-Based Analytical Interfaces

The analytical web dashboard exposes the trained models through interactive user-facing interfaces, as shown in Fig. 5.

The fare-estimation interface in Fig. 5a allows a user to configure trip characteristics and obtain a predicted fare. The illustrated example evaluates a 24.2-mile trip from Midtown Center to JFK Airport at 18:00 with one passenger and clear weather, producing an estimated fare of $134.03.

The demand interface in Fig. 5b accepts a target taxi zone and observation hour and presents the corresponding demand-regime classification together with a GIS visualization.

F. Speed Layer Real-Time Behaviour & Dashboard Telemetry

The Structured Streaming engine processes approximately 200–250 replayed trip events every 3 seconds. In the project measurements, average micro-batch processing time is approximately 1.12 seconds, remaining below the 3-second trigger interval. A 10-minute event-time watermark is used to control the retained state for windowed aggregation.

The WebHDFS bridge updates local JSON telemetry snapshots approximately every 2 seconds. The corresponding streaming interfaces are shown in Fig. **??**.

The live dashboard provides three principal streaming views:

- **Live Streaming Telemetry (Fig. 6a):** Displays stream status, timestamps, trip activity, active zones, predicted fares, actual fares, and live residual metrics.
- **Real-Time GIS Mapping (Fig. 6b):** Displays pickup locations and active demand information geographically.
- **Stateful Windowed Regimes (Fig. 6c):** Displays trailing 1-hour pickup aggregations and their associated Low-, Moderate-, or High-Demand classifications.

G. Discussion of Design Choices

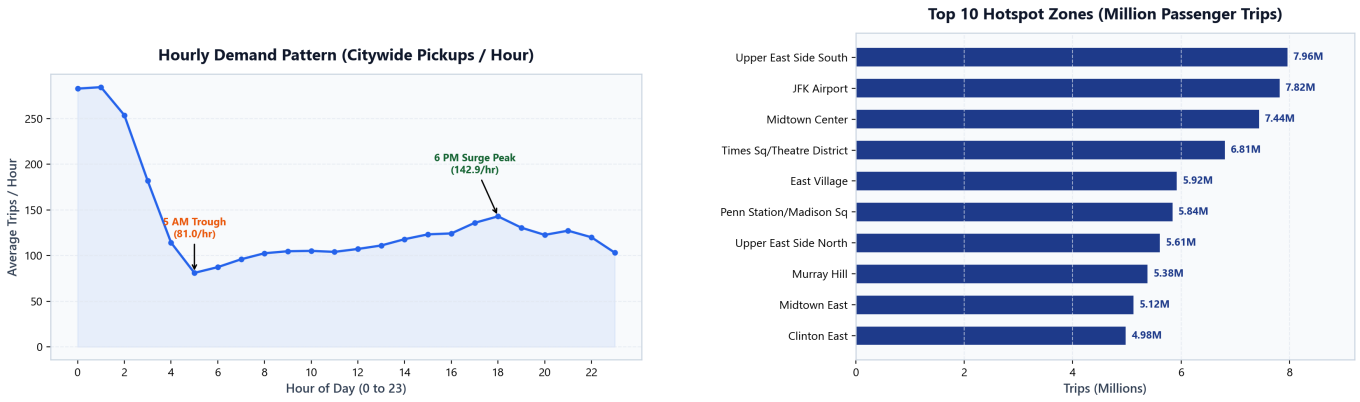
Ridge Regression was selected because the fitted model provides inexpensive pointwise inference while L_2 regularization constrains the coefficient vector. The model can therefore be integrated directly into a high-throughput streaming pipeline without traversing a large ensemble of decision trees.

The $\log(1+x)$ transformation was selected for the demand-clustering pipeline because pickup counts are highly skewed. Compressing the upper tail before Euclidean-distance-based clustering reduces the influence of exceptionally high-volume observations and produces more separable operational demand regimes.

VIII. Limitations

The platform has several engineering limitations:

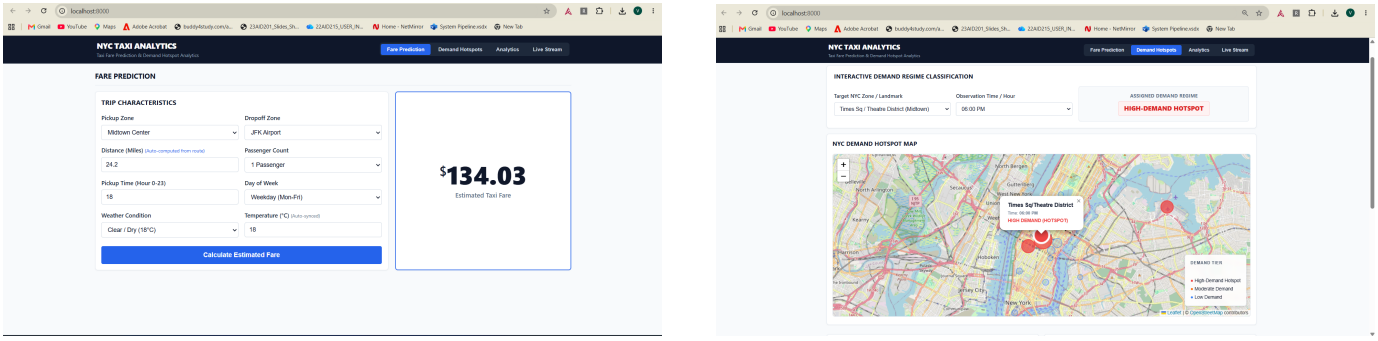
1) **Long-Haul Toll Variance:** Long-distance trips exhibit larger residuals because tolls and certain airport-related charges are not fully represented by the pre-ride feature set.
2) **Pre-Ride Target Leakage Boundaries:** Variables such as `tolls_amount`, `tip_amount`, and `extra` were excluded from the predictor set because they can contain information that is unavailable or inappropriate for a true pre-ride prediction setting.
3) **Simulated Streaming Ingestion:** Streaming micro-batches are replayed from held-out historical records rather than originating from a live commercial taxi telemetry service.
4) **Deployment Scope:** Although HDFS storage is distributed across the configured DataNodes, the exper-



(a) 24-hour diurnal demand curve showing the early-morning trough and evening peak.

(b) Top 10 highest-volume pickup zones in the project dataset.

Fig. 4: Spatiotemporal demand characteristics derived from the historical trip dataset: (a) hourly demand behaviour and (b) the highest-volume pickup zones.



(a) Tab 1: Interactive Fare Prediction Calculator.

(b) Tab 2: Interactive Demand Regime Classification and Spatial Hotspot Map.

Fig. 5: Locally hosted analytical dashboard interfaces for pre-ride fare estimation and spatial demand-regime exploration.

imental compute environment does not represent a geographically distributed production Spark deployment.

5) **Static Spatial Boundaries:** Taxi zones are treated as discrete spatial regions and therefore do not capture all street-level routing variation within a zone.

IX. Conclusion and Future Work

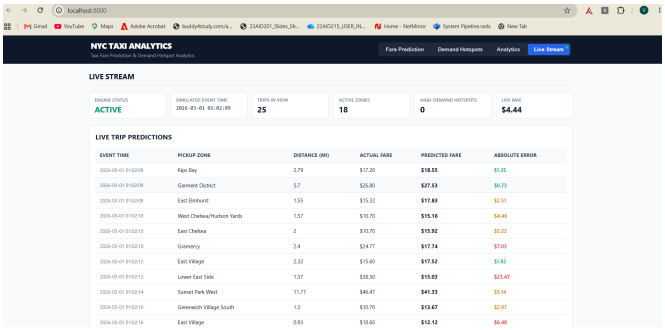
This paper presented an end-to-end Lambda-style big data analytics platform for taxi fare prediction and demand hotspot analytics using Apache Spark and Hadoop HDFS. In the batch layer, 33.5 Million trip records were cleaned, enriched with weather and spatial information, and processed for machine-learning workloads.

The tuned L-BFGS Ridge Regression model achieved $R^2 = 0.799821$, RMSE = \$7.7889, and MAE = \$4.4027 on the held-out test set, with 26.3034% of predictions lying within $\pm$\$2.00 of the corresponding metered fare.

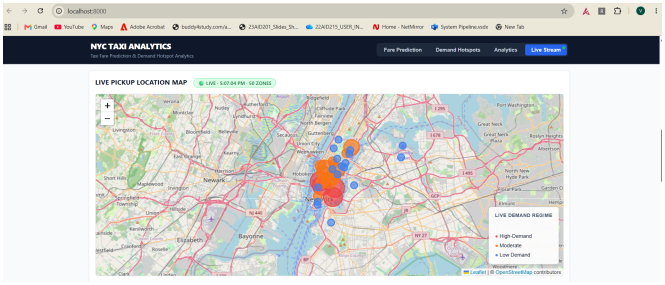
For demand analysis, the $\log(1 + x)$ transformed K-Means pipeline achieved a peak Silhouette score of 0.6399 at $K = 3$, compared with 0.3898 for the baseline representation.

The speed layer extends the batch pipeline through Spark Structured Streaming, combining pointwise fare inference with stateful sliding-window demand aggregation. Results are served from HDFS through a WebHDFS bridge to a locally hosted interactive dashboard.

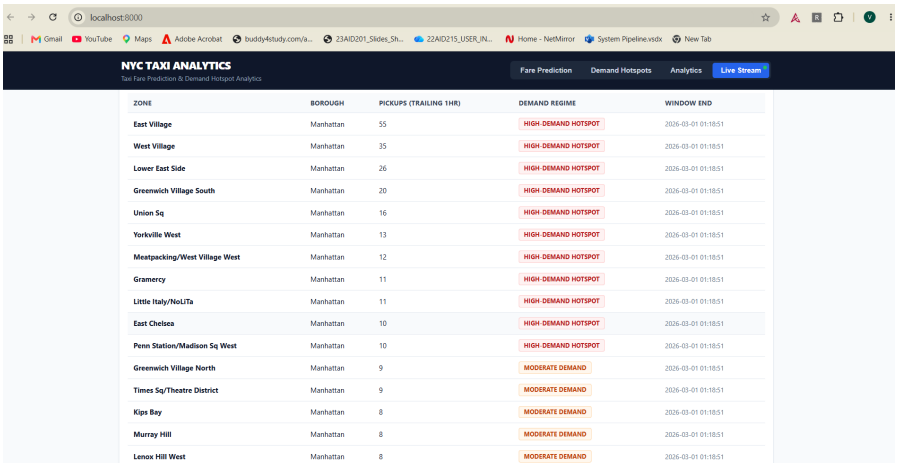
Future work can extend the system through dynamic toll information, richer road-network features, graph-based spatial models, Apache Kafka-based ingestion, and multi-worker distributed Spark evaluation.



(a) Streaming telemetry and live fare-prediction feed.



(b) Live pickup-location and demand map.



(c) Stateful windowed demand-regime table.

Fig. 6: Real-time streaming telemetry interface showing pointwise fare inference, live spatial activity, and stateful sliding-window demand regimes.

REFERENCES

[1] New York City Taxi and Limousine Commission, "TLC Trip Record Data," New York, NY, USA. [Online]. Available: https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page. Accessed: Oct. 19, 2026.

[2] B. S. B. D. Kumar and S. Hetharion, "Scalable Big Data Analytics and Fare Prediction for NYC Taxi Trips Using Distributed Computing and Machine Learning," *International Journal of Information Engineering and Science*, vol. 3, no. 1, pp. 1–12, 2026. doi: 10.62951/ijies.v3i1.97.

[3] K. Sundravadivelu, K. Karthigadevi, M. Sakthimohan, G. Elizabeth Rani, M. Thenmozhi, and V. Sanjeevi Kumar, "Taxi Fare Prediction using Random Forest: A Data-Driven Approach based on Pickup/Dropoff Locations and Ride Attributes," in *Proc. 7th International Conference on Intelligent Sustainable Systems (ICISS)*, 2025, pp. 969–974. doi: 10.1109/ICISS63372.2025.11076256.

[4] S. Agarwal, A. Aggarwal, T. Joshi, R. Khare, and P. Gupta, "Taxi Fare Prediction Using Various Machine Learning Models," *International Journal for Multidisciplinary Research*, vol. 7, no. 1, 2025. doi: 10.36948/ijfmr.2025.v07i01.35839.

[5] T. Ramadhan, H. Ariiq, M. D. Arjun, and M. R. A. Aditya, "Application of K-Means Clustering for Urban Transportation Pattern Analysis Using Big Data Trip Dataset," *Journal of Artificial Intelligence and Engineering Applications*, vol. 5, no. 3, pp. 3721–3729, 2026. doi: 10.59934/jaiea.v5i3.2237.

[6] S. Mishra and S. Khetarpaul, "Identifying and recommending taxi hotspots in spatio-temporal space," *GeoInformatica*, vol. 29, no. 1, pp. 93–113, 2025. doi: 10.1007/s10707-024-00524-w.

[7] K. Shvachko, H. Kuang, S. Radia, and R. Chansler, "The Hadoop Distributed File System," in *Proc. IEEE 26th Symposium on Mass Storage Systems and Technologies (MSST)*, 2010, pp. 1–10. doi: 10.1109/MSST.2010.5496972.

[8] M. Zaharia, M. Chowdhury, T. Das, A. Dave, J. Ma, M. McCauley, M. J. Franklin, S. Shenker, and I. Stoica, "Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing," in *Proc. 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2012, pp. 15–28. [Online]. Available: https://www.usenix.org/conference/nsdi12/technical-sessions/presentation/zaharia.

[9] M. Armbrust, R. S. Xin, C. Lian, Y. Huai, D. Liu, J. K. Bradley, X. Meng, T. Kaftan, M. J. Franklin, A. Ghodsi, and M. Zaharia, "Spark SQL: Relational Data Processing in Spark," in *Proc. ACM SIGMOD International Conference on Management of Data*, 2015, pp. 1383–1394. doi: 10.1145/2723372.2742797.

[10] M. Armbrust, T. Das, J. Torres, B. Yavuz, S. Zhu, R. Xin, A. Ghodsi, I. Stoica, and M. Zaharia, "Structured Streaming: A Declarative API for Real-Time Applications in Apache Spark," in *Proc. ACM SIGMOD International Conference on Management of Data*, 2018, pp. 601–613. doi: 10.1145/3183713.3190664.

[11] X. Meng, J. Bradley, B. Yavuz, E. Sparks, S. Venkataraman, D. Liu, J. Freeman, D. B. Tsai, M. Amde, S. Owen, D. Xin, R. Xin, M. J. Franklin, R. Zadeh, M. Zaharia, and A. Talwalkar, "MLlib: Machine Learning in Apache Spark," *Journal of Machine Learning Research*, vol. 17, no. 34, pp. 1–7, 2016. [Online]. Available: https://www.jmlr.org/papers/v17/15-237.html.

[12] N. Marz and J. Warren, *Big Data: Principles and Best Practices of Scalable Real-Time Data Systems*. Shelter Island, NY, USA: Manning Publications, 2015. [Online]. Available: https://www.manning.com/books/big-data.

[13] D. C. Liu and J. Nocedal, "On the Limited Memory BFGS Method for Large Scale Optimization," *Mathematical Programming*, vol. 45, pp. 503–528, 1989. doi: 10.1007/BF01589116.

[14] Open-Meteo, "Historical Weather API," [Online]. Available: https://open-meteo.com/en/docs/historical-weather-api. Accessed: Oct. 19, 2026.